IMBALANCED DATA

Data Mining I – TSD 2022

Big amounts of data

With the advent of information technologies the amount of data that is generated everyday is growing at a fast pace.

Trying to extract information and knowledge from that vast cumulus of data is a time consuming (if not impossible) task to do by hand.

Four possibilities that can be used for solving imbalance data problem

- ✓ Choice of appropriate classifier
- ✓ Use cost sensitive approach
- ✓ Use sampling based approach
- ✓ Bagging

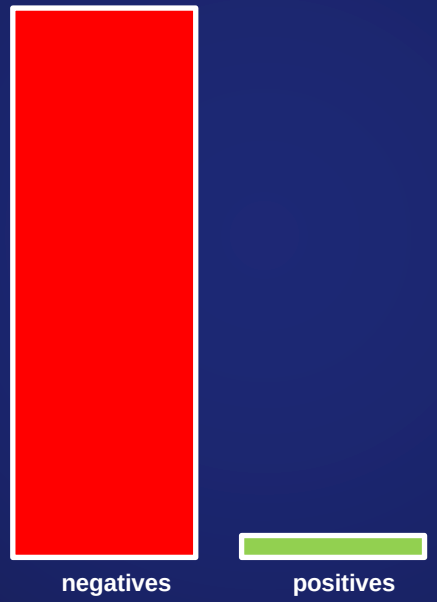
You trained a model to predict cancer from image data using a state of the art Hierarchical siamese CNN with dynamic kernel activations...

Your model has an accuracy of 99.9%

But... WTH!?

By looking at the confusion matrix you realize that the model does not detect any of the positive examples.

After plotting your class distribution you see that you have thousands of negative examples but just a couple of positives.



Classifiers try to reduce the overall error so they can be biased towards the majority class.

Negatives = 998
Positives = 2

By always predicting a negative class the accuracy will be 99.8% !!

Your dataset is imbalanced.

Now what??

What can you do?

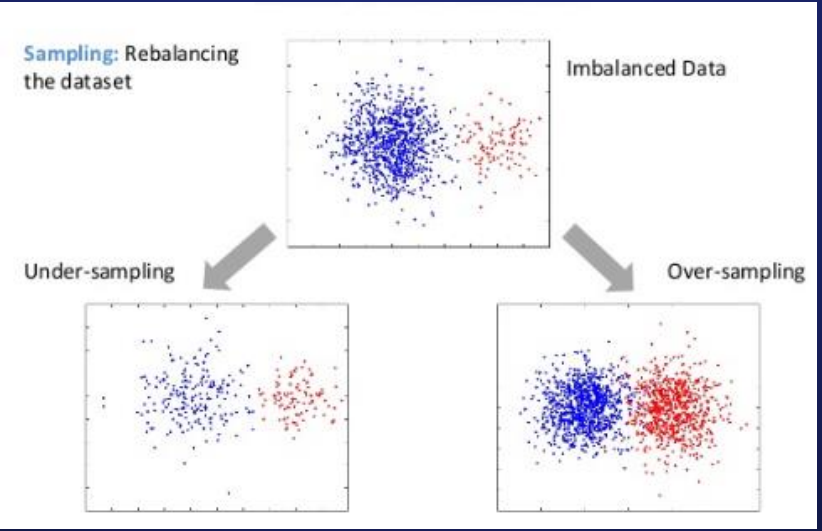
- Collect more data (difficult in many domains)
- Delete data from the majority class
- Create synthetic data
- Adapt your learning algorithm (cost sensitive classification)

Main idea of sampling based approach is to modify the distribution of events so that the rare class is well represented in the training sample.

There are:
Undersampling
Oversampling
Hybrid oversampling and undersampling

Random over/under sampling

- Random *oversampling*: randomly duplicate data points from the minority class.
- Random undersampling: randomly delete data points from the majority class.



Oversampling is replication the events of minority class.

Potential problem: could be for this method is overfitting for noisy data, because noisy data will be replicate.

To avoid overfitting the procedure of randomized oversampling is proposed (SMOTE and Borderline-SMOTE) with cleaning noisy data.

Hui Han, Wen-Yauan Wang, Bing-Huan Mao, Bodeline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning, ICIC 2005, part 1, LNCS 3644, 878-887, 2005.

In case of undersampling we can take random sample of majority class.

Potential problem : some of useful instances may not be chosen for training and classifier will not be optimal.

Reduction majority class without losing performance of classification can be used.

Problems with these approaches:

- Loss of information (in the case of under sampling)
- Overfitting and fixed boundaries (over sampling)

UNDERSAMPLING : Random Under Sampling (RUS), K-means Clustering Centroids (CC), Neighborhood Cleaning Rule (NCL), One-sided Selection (OSS), Nearmiss-1, Nearmiss-2, and Tomek Links

OVERSAMPLING : Random Over Sampling (ROS) and Synthetic Minority Oversampling Technique (SMOTE)

HYBRID : SMOTE + Tomek and SMOTE + ENN (Edited Nearest Neighbors)

Over Sampling Algorithms based on SMOTE

1-SMOTE: Synthetic Minority Over sampling Technique (SMOTE)
2- ADASYN: ADAptive SYNthetic
3- ANS: Adaptive Neighbor Synthetic
4- Borderline SMOTE
5-Safe Level SMOTE
6- DBSMOTE: Density-Based Synthetic Minority Over-sampling Technique

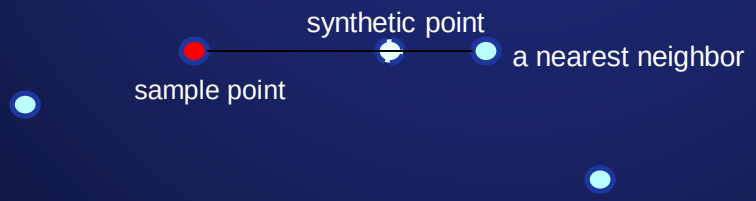
SMOTE

- Synthetic Minority Over-sampling Technique (Chawla).
- Creates new data points from the minority class.
- Operates in the feature space.

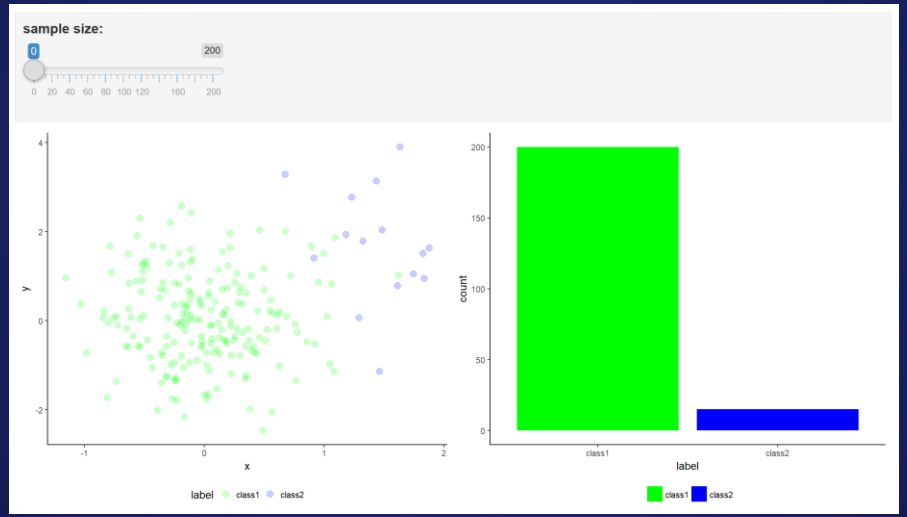
- Main steps:

1. Take the difference between a sample point and one of its nearest neighbors.
2. Multiply the difference by a random number between 0 and 1 and add it to the feature vector.

This causes the selection of a random point along the line segment between two specific features.



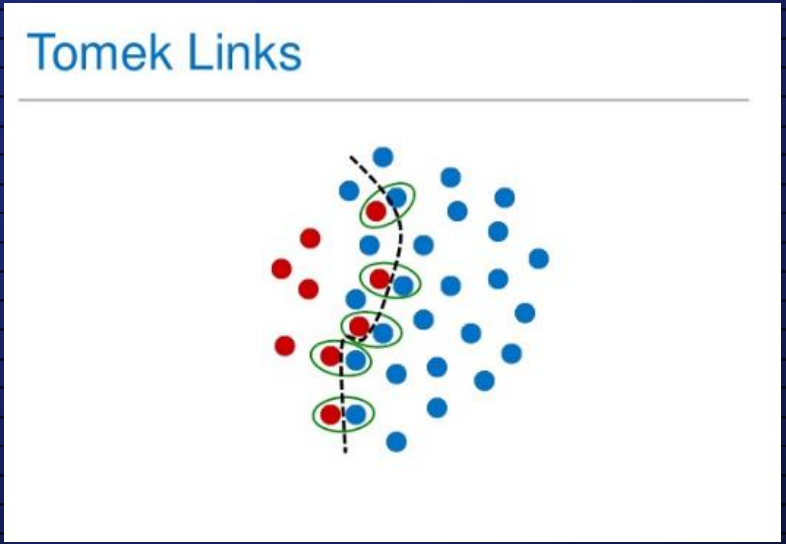
SMOTE DEMO


sample size:
0
200
0 20 40 60 80 100 120 160 200
y
x
label
class1
class2
count
label
class1
class2

Hybrid oversampling and undersampling

1-SMOTE-Tomek

2-SMOTE-ENN (Edited Nearest Neighbor)



Danger of information injection and overfitting

Do not create synthetic points on the entire dataset before splitting into train/test sets.

- Perform the preprocessing just on the training data!!
- For k-fold cross validation, you have to do it for each fold (just on the training set).

For images:

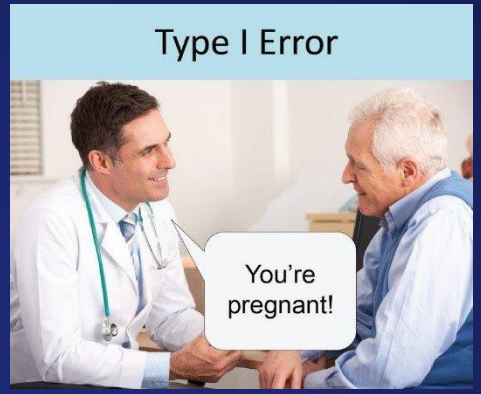
- Augment training data by applying image transformations: rotate, scale, shift, etc.

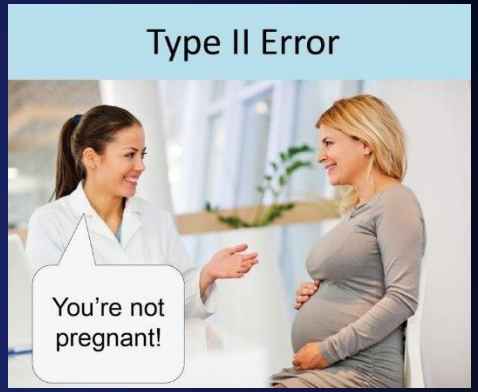
- Keras provides functionalities for data augmentation: https://blog.keras.io/building-powerful-image-classification-models-using-very-little-data.html

Cost-sensitive learning

Motivation:
Data with different misclassification costs.

Objective: Minimize the total misclassification cost.





- Based on the classifier predicted probabilities.
- Binary case: predict positive if probability is > 0.5
- Probability threshold can be changed using a cost matrix:

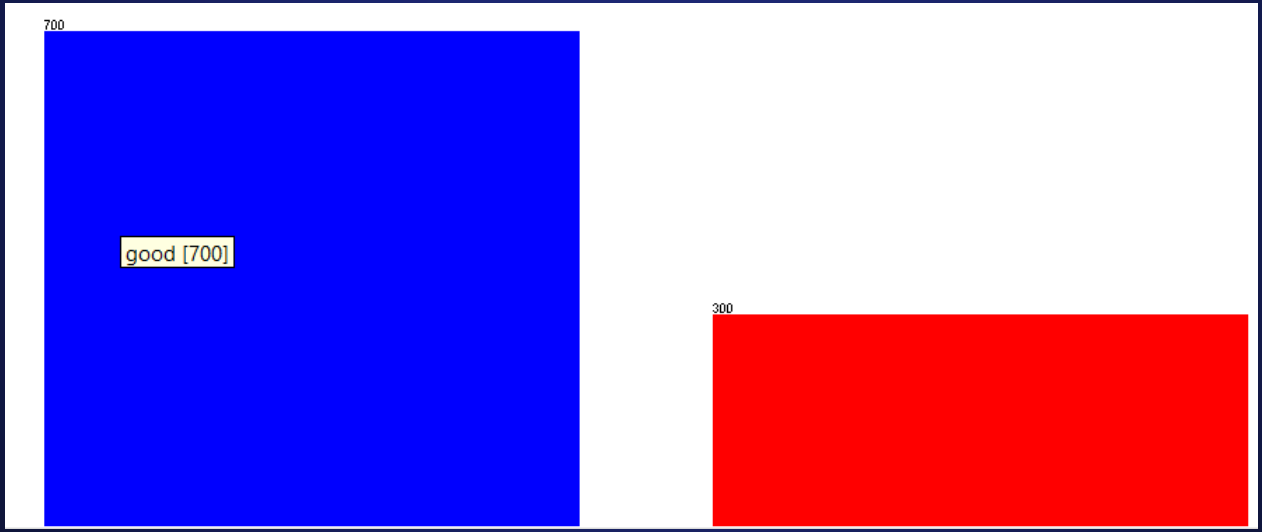
TP: true positives → 0 ; **FN: false negatives** → λ

FP: false positives → μ ; **TN: true negatives** → 0

Classify as positive if: probability of positive > $\frac{\mu}{\mu+\lambda}$

Cost-sensitive classification

Credit dataset.



Cost-sensitive classification with Weka

- Naïve Bayes.

```
=== Summary ===

Correctly Classified Instances         754               75.4    %
Incorrectly Classified Instances       246               24.6    %
Kappa statistic                          0.3813
Mean absolute error                      0.2936
Root mean squared error                  0.4201
Relative absolute error                 69.8801 %
Root relative squared error             91.6718 %
Total Number of Instances             1000

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.864    0.503    0.800      0.864    0.831      0.385    0.787     0.891     good
                 0.497    0.136    0.611      0.497    0.548      0.385    0.787     0.577     bad
Weighted Avg.    0.754    0.393    0.743      0.754    0.746      0.385    0.787     0.797

=== Confusion Matrix ===

   a   b   <-- classified as
 605  95 |   a = good
 151 149 |   b = bad
```

TP

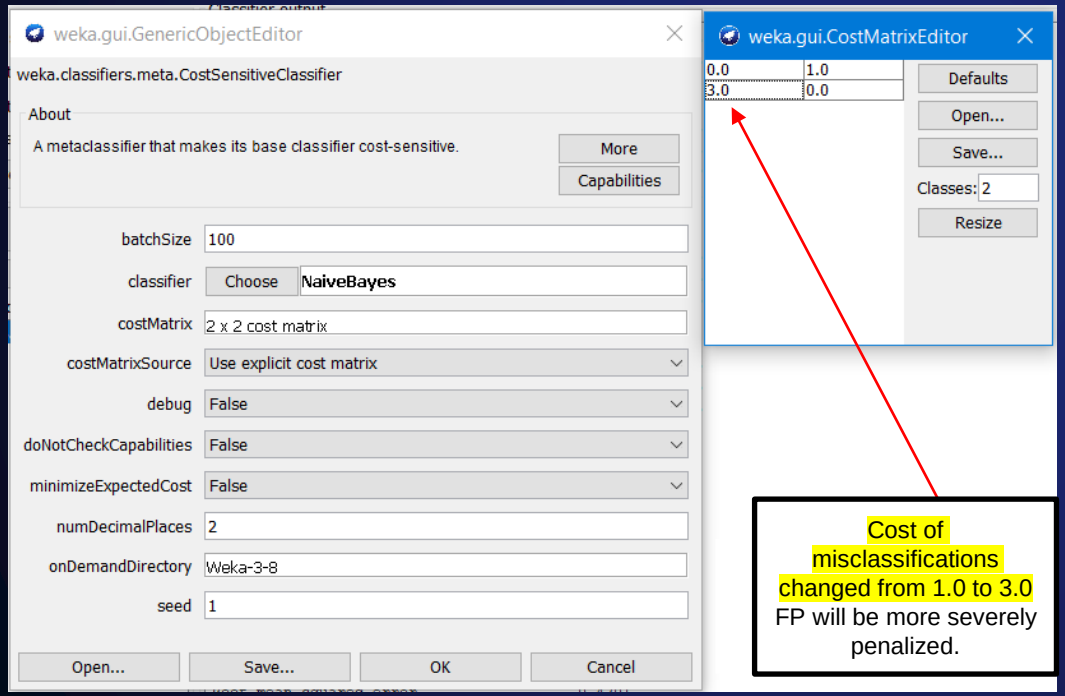
FN

FP

TN

Cost-sensitive classification with Weka

- Cost-sensitive classifier with Naïve Bayes.



$$\frac{\mu}{\mu+\lambda} = \frac{3}{3+1} = 0.75$$

Cost-sensitive classification with Weka

- Cost-sensitive classifier with Naïve Bayes.

```
=== Summary ===

Correctly Classified Instances         720               72      %
Incorrectly Classified Instances       280               28      %
Kappa statistic                          0.4078
Mean absolute error                      0.3367
Root mean squared error                  0.4448
Relative absolute error                 80.1422 %
Root relative squared error             97.068  %
Total Number of Instances             1000

=== Detailed Accuracy By Class ===

                 TP Rate  FP Rate  Precision  Recall   F-Measure  MCC      ROC Area  PRC Area  Class
                 0.706    0.247    0.870      0.706    0.779      0.425    0.787     0.891     good
                 0.753    0.294    0.523      0.753    0.617      0.425    0.787     0.578     bad
Weighted Avg.    0.720    0.261    0.766      0.720    0.731      0.425    0.787     0.797

=== Confusion Matrix ===

   a   b   <-- classified as
 494 206 |   a = good
  74 226 |   b = bad
```

But also TP reduced

FPs reduced

Assign class weights

- In Keras you can assign class weights.

fit() function in keras has a class_weight parameter (see https://keras.io/models/model/).

class_weight = {0: 1.,
1: 9.}

Optional dictionary mapping class indices (integers) to a weight (float) value, used for weighting the loss function (during training only). This can be useful to tell the model to "pay more attention" to samples from an under-represented class

max_v = number of points with majority class
weight_minority = max_v / (# of minority points)
weight_majority = max_v / max_v = 1

This is just one way to compute class weights

Tools

- Python imbalanced-learn library: https://github.com/scikit-learn-contrib/imbalanced-learn
- Weka also has oversampling methods and a cost sensitive meta classifier: https://weka.wikispaces.com/CostSensitiveClassifier

Performance metrics

- Most of the time accuracy will not be enough to assess performance.
- $accuracy = \frac{TP+TN}{P+N}$ Percentage of correctly classified instances.
- $sensitivity = \frac{TP}{P}$ The proportion of positives that are correctly identified as such.
- $precision = \frac{TP}{TP+FP}$ Equivalently, it is the fraction of relevant instances among the selected ones.

$$\mathrm{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Matthews correlation coefficient (takes into account imbalance)

Supporting materials: imbalanced data

- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. Journal of artificial intelligence research, 16, 321-357.
- Kotsiantis, S., Kanellopoulos, D., & Pintelas, P. (2006). Handling imbalanced datasets: A review. *GESTS International Transactions on Computer Science and Engineering*, *30*(1), 25-36.

- SMOTE: https://www.datasciencecentral.com/handling-imbalanced-data-sets-in-supervised-learning-using-family/
- Python imbalanced-learn library: https://github.com/scikit-learn-contrib/imbalanced-learn
- Weka also has oversampling methods and a cost sensitive meta classifier: https://weka.wikispaces.com/CostSensitiveClassifier
- Cost-sensitive classification video: https://www.youtube.com/watch?v=l9muPldOG30
- Performance metrics: https://en.wikipedia.org/wiki/Sensitivity_and_specificity

Thanks!

Do you have any questions?

Credits: This presentation template was created by **Slidesgo**, including icons by **Flaticon**, infographics & images by **Freepik**